2019 EXAMINATIONS

Lancaster University

Part II

COMPUTING AND COMMUNICATIONS [2.5 Hours]

SCC.311 Distributed Systems

Candidates are asked to answer THREE questions from FOUR; each question is worth a total of 25 marks.

Use a separate answer book for each question.

Question 1

1.a In indirect communication, space uncoupling is sought to support mobility.

i. Explain what the notion of space uncoupling involves.

[3 marks]

ii. Consider an email application that only allows the sending of 1-to-1 messages; i.e. no mailing lists. Taking the perspective of the developer, describe how space uncoupling facilitates having mobile clients access this email system (3 marks). Use a diagram to illustrate the different elements of your system, indicating the type of indirect communication approach that is most suitable for this system (3 marks).
Hint: Mobile clients (e.g. smartphones) change their network address as they move from one network access point to another.

[6 marks]

iii. Identify and describe a state-based indirect communication approach that provides *both* space- and time-coupling.

[1 mark]

1.b You are deploying a 3-layer (presentation, business logic, data) application in the cloud. Describe one of the benefits of having such layered architecture.

[2 marks]

1.c Data stored in the Google File System (GFS) is automatically replicated.

i. Describe how this works -- i.e. what is stored and where? Assume GFS default settings.

[3 marks]

ii. Comment briefly on the rationale behind the above design.

[2 marks]

1.d Scalability is the ability to deal with significant change in a certain dimension of the system. Identify and explain any one of those dimensions using any example application.

[4 marks]

1.e Describe the mediator architectural design pattern.
Identify and describe what type of transparency it provides, if any.

[4 marks]

Total 25 marks

Question 2

2.a Briefly explain how a group communication service operates? (3 marks) Provide one example of how group communication might be useful in distributed systems (2 marks).

[5 marks]

2.b Consider the following pseudo code that implements a group multicast operation

```
void multicast(Message msg) {
        for each member in the group {
                ack = FALSE;
                while(ack != TRUE) {
                        ack = send(member, msg);
                }
        }
}
```

Here***, send*** is a communication primitive which does not provide any delivery guarantee.

Which level of *reliability* does this code provide? (1 mark)

What form of fault does this level of reliability try to address? (2 marks)

[3 marks]

Question 2 continues on next page…

Question 2 continued.

2.c A group communication library provides the following operations, *join*, *multicast* and *leave*. Consider the following pseudo-code built upon this group communication service. *multicast* (line 5) sends a message to all participants of a group. When a message arrives, the *onReceive* method (lines 13-115) will be invoked by the library.

Consider the following implementation using this group communication middleware:

```
1   static void main () {
2         join ("SCC.311");
3
4         for (i=0; i<4; i++)
5               multicast("Hello World! ");
6         }
7
8         sleep(1000); //sleep for 1000 ms
9
10        leave("SCC.311");
11  }
12
13  void onReceive(String msg) {
14      print (msg);
15  }
```

i. What type of message reception mode does this code implement?

[1 mark]

ii. Assuming *five (5)* instances of this program are launched at the same time:

What is the maximum number of "Hello World!" messages to be printed out? Briefly explain your answer (4 marks)
What is the minimum number of "Hello World!" message to be printed out? Briefly explain your answer (2 marks)

[6 marks]

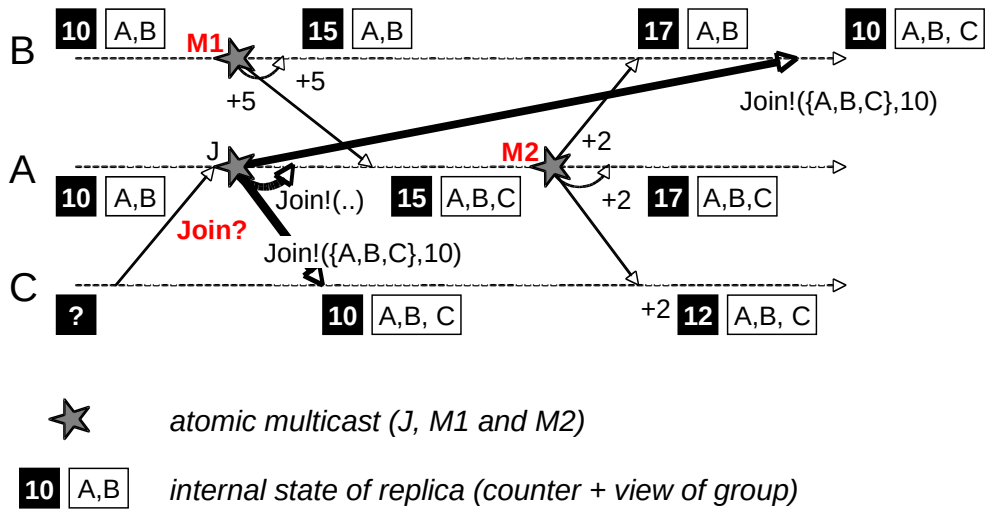
Question 2 continues on next page…

Question 2 continued.

2.d

i. A local bank wants to replicate one of their services with an active replication scheme. This service only needs to support crediting operations (i.e. additions, which are commutative). They plan to use a multicast protocol to disseminate incoming requests across a fixed set of replicas, and do not envisage a need to dynamically deploy new replicas. Do they need any ordering guarantees to insure both replicas behave consistently? Briefly explain your answer, clearly state your assumption. **[3 marks]**

ii. This bank now decides to support dynamic deployment of new replicas to provide better load balance. Their new implementation uses *atomic* multicast to propagate messages and to notify new group members (i.e., new replicas). The system administrator observes the following scenario soon after the new system went online.



At the start of this scenario, the communication group has two replicas, ***A*** and ***B***, both have the same internal state (***10***), and the same view of the group membership (***A, B***). A new replica ***C*** is added by sending a join request ("Join?") to ***A*** to join the group. ***A*** multicasts its new view of the group (***A, B, C***) along with its internal state to help ***C*** to initialise itself. In parallel ***B*** multicasts a client request that it has just received (***M1 = "+5"***). Later, ***A*** does the same with another request (***M2 = "+2"***). Unfortunately, at the end of the run, all replicas have reached different internal states, which is catastrophic.

Explain why this problem has occurred (3 marks)

Explain how this can be solved by an additional group communication property (4 marks)

[7 marks]

Total 25 marks

Question 3

3.a Remote invocation makes use of an Interface Definition Language (IDL).

i. Describe what an IDL is.

[2 marks]

ii. Why is it useful to define the interfaces of an object in an IDL? Give two reasons.

[4 marks]

iii. Remote Procedure Call (RPC) and Remote Method Invocation (RMI) are two models of remote invocation. Explain what RMI offers that RPC does not.

[3 marks]

3.b Consider a file server that only provides 2 simple functionalities: `listDirectories` and `listFiles`, both of which read from a file structure that is already created and stored on the server's disk. They do not change the existing file structure.
In terms of performance, would it be better to build this server as stateful or stateless? Explain your answer.

[3 marks]

3.c Describe with an example what location transparency is.
Explain what type of uncoupling it provides.

[3 marks]

3.d MapReduce is designed with 3 key principles. Identify and briefly describe any <u>one</u> of them.

[2 marks]

3.e You are hired as a Teaching Assistant for the SCC.311 module. You are asked to use the private cloud services provided by ISS to develop isolated runtime environments for the next cohort of SCC.311 students. Each student will get their own runtime environment that comes preconfigured with the Java compiler and some additional libraries that are needed for the coursework. ISS will provide you with virtual machines each with a standard Linux operating system.

i. What level of cloud services is ISS providing to you? Explain your answer. **[4 marks]**

ii. What level of cloud services would you be providing to the students? Explain your answer.

[4 marks]

Total 25 marks

Question 4

4.a Explain what graceful degradation is and its purpose.

[3 marks]

4.b You are asked to review a core function of a replication middleware. The pseudo code of this function is given as follows:

```
OnReceive(client_request) {
  if (I_am_the_primary) {
            process client_request
            send result to client
            for each of the 4 remaining replicas {
                 send update to the replica
            }
   } else {
            inform client who the primary is
   }
}
```

In the following questions we assume duplicated client requests are properly handled by the middleware, and that there is a mechanism to safely switch to a new primary server if the current primary crashes. We also assume only one client can connect to the middleware service at any time.

Question 4 continues on next page…

Question 4 continued.

4.b

i. One of your friends suggests using an *atomic multicast* operation to replace the *for loop at line 5*. This revised pseudo code is given as follows

```
1  on_receive(client_request) {
2     if (I_am_the_primary_server == true) {
3              process client_request
4              send result to client
5              atomic_multicast(remaining_replicas[4], update)
6     } else {
7         inform client who the primary is
8     }
9  }
```

Do you think this modification is necessary? (1 mark) Briefly explain your answer (2 marks)
Illustrate your explanations with an example using a time-sequence diagram. (2 marks)

[5 marks]

ii. Even with your friend's modification, there is still a problem with the code presented above (4.b.i).

Describe a scenario (using a time-sequence diagram) where *the primary crashes and the client receives an incorrect answer* (4 marks).
Briefly describe how you will fix this problem. (2 marks)

Bear in mind here we assume there is only one client at a time, and duplicated requests are properly handled.

[6 marks]

Question 4 continues on next page…

Question 4 continued.

4.c A distributed system uses two-phase commit to implement *atomic* transactions. As a reminder, two-phase commit works as follow:

- The coordinator sends a **canCommit** request to each participant;
- Each participant replies with **YES** or **NO**, according to whether the participant can commit or not;
- If all participants answer **YES**, the coordinator sends **doCommit** to all participants, else the coordinator sends **doAbort** to participants that replied **YES**.

i. Suppose a coordinator sends a **doCommit** message to some participants of a transaction, but it crashes before sending any more messages. Explain what the coordinator must do after it reboots.

[4 marks]

ii. Now suppose a client has asked coordinator ***A*** to run two-phase commit for a transaction. Ten minutes later, the client has received no response for ***A*** and is unable to contact ***A***. Explain that why the client should *NOT* find a new coordinator ***B*** and asks ***B*** to run two-phase commit again for the same transaction (by starting from the **canCommit** request).

[7 marks]

Total 25 marks

--- End of Paper ---